

Virexo: Low-Token Antigravity Build Plan

Part 1 — Initial Antigravity Setup

1. Create an empty GitHub repository named `virexo`.
2. Clone it locally.
3. Open the repository in Antigravity.
4. Open the Agent panel.
5. Click `...`.
6. Open `Customizations`.
7. Select `Rules`.
8. Click `+ Workspace`.
9. Name the rule `virexo-project`.
10. Set activation to `Always On`.
11. Paste the rule below.

Part 2 — Always-On Workspace Rule

```
# Virexo Project Contract
```

```
You are the senior engineer responsible for Virexo, a production-quality MERN  
real-time chat application.
```

```
## Stack
```

```
Frontend:
```

- React 19
- Vite
- Tailwind CSS
- React Router
- Zustand
- TanStack Query
- Axios
- Socket.io Client
- Framer Motion

```
Backend:
```

- Node.js
- Express
- MongoDB Atlas
- Mongoose
- Socket.io
- JWT access tokens

- Rotating refresh tokens
- bcrypt
- cookie-parser
- Multer
- Cloudinary
- Helmet
- express-validator
- express-rate-limit

Testing:

- Vitest
- React Testing Library
- Supertest
- Playwright where valuable

Deployment:

- Frontend: Vercel Hobby
- Backend: Render Free Web Service
- Database: MongoDB Atlas Free
- Media: Cloudinary Free
- Transactional email: Brevo Free through an email adapter

Use only free services and open-source packages.

Architecture

Use an npm-workspaces monorepo:

- apps/web
- apps/api
- packages/shared
- docs

Use feature-based modules and clear separation between routes, controllers, services, repositories/models, validation and middleware.

Do not add abstraction without a present requirement.

Use JavaScript ES modules consistently.

Security

- Never commit secrets.
- Validate all external input.
- Sanitize user-controlled data.
- Use short-lived access tokens.
- Keep access tokens in application memory, not localStorage.
- Store refresh tokens in secure HttpOnly cookies.

- Rotate refresh tokens and detect token reuse.
- Store only hashed refresh tokens.
- Apply authorization at service/API boundaries.
- Use secure CORS and cookie configuration.
- Restrict uploads by MIME type and size.
- Upload media using memory streams; never depend on Render local storage.
- Prevent mass assignment and sensitive-field leakage.
- Use generic authentication errors where appropriate.
- Add rate limits to authentication, search, upload and report endpoints.

UI

- Mobile-first.
- Responsive desktop layout.
- Light and dark themes.
- Accessible contrast and keyboard navigation.
- Premium Discord/Telegram/Linear-inspired appearance.
- Reusable design tokens and components.
- Avoid excessive glassmorphism and animation.
- Respect prefers-reduced-motion.
- Include loading, empty, error and skeleton states.

Execution Contract

For every task:

1. Read this rule, repository files and ``docs/BUILD_STATUS.md``.
2. Inspect existing code before editing.
3. Implement only the requested phase.
4. Do not rewrite unchanged files.
5. Do not implement future phases.
6. Modify files directly instead of printing full source code in chat.
7. Keep the smallest coherent implementation.
8. Run lint, tests and builds relevant to the changes.
9. Fix failures caused by the phase.
10. Update ``docs/BUILD_STATUS.md``.
11. Do not create a Git commit automatically.
12. Stop after the phase.

Your final response must contain only:

1. Goal
2. Architecture Decisions
3. Changed Files
4. Manual Work
5. Commands Run
6. Test Results
7. Common Errors

8. Git Commit Message
9. Next Phase

Keep the final response below 500 words. Do not paste code unless I explicitly ask.

Part 3 — Reusable Workflow

In Antigravity:

1. Open `...`.
2. Open `Customizations`.
3. Select `Workflows`.
4. Click `+ Workspace`.
5. Name it `virexo-phase`.
6. Paste:

```
---
title: Virexo Phase
description: Implements one controlled Virexo development phase.
---

1. Read the Always On Virexo workspace rule.
2. Read `docs/BUILD_STATUS.md`.
3. Inspect relevant existing files.
4. Restate the requested phase internally as acceptance criteria.
5. Create an implementation plan before changing code.
6. Implement only the requested scope.
7. Run relevant lint, tests and builds.
8. Use browser verification when a UI is involved.
9. Fix failures introduced by this phase.
10. Update `docs/BUILD_STATUS.md`.
11. Return only the response structure required by the workspace rule.
12. Stop.
```

Use a **new Antigravity conversation for every phase**. This prevents old conversation history from consuming tokens.

Phase Prompts

Phase 0 — Product Specification

```
/virexo-phase
```

Phase 0: create the Virexo technical specification only.

Create:

- docs/PRODUCT_REQUIREMENTS.md
- docs/ARCHITECTURE.md
- docs/DATABASE_SCHEMA.md
- docs/API_CONVENTIONS.md
- docs/SECURITY_MODEL.md
- docs/BUILD_STATUS.md

Include Mermaid architecture and authentication sequence diagrams.

Define MVP, post-MVP and admin scope. Resolve feature conflicts and document assumptions. Do not create application code.

Acceptance:

- Requirements are testable.
- Data relationships and indexes are defined.
- REST and Socket.IO responsibilities are separated.
- Free-tier limitations are documented.

Commit:

```
git add .  
git commit -m "docs: define Virexo product and system architecture"
```

Phase 1 — Monorepo Foundation

```
/virexo-phase
```

Phase 1: create the runnable npm-workspaces monorepo foundation.

Create:

- apps/web using React 19 and Vite
- apps/api using Express ES modules

- packages/shared
- root workspace scripts
- ESLint and Prettier configuration
- environment example files
- API GET /health endpoint
- basic frontend health page

Do not add authentication or database models.

Acceptance:

- One root command starts web and API.
- Lint succeeds.
- Frontend production build succeeds.
- API health endpoint returns JSON.

Commit:

```
git add .  
git commit -m "chore: initialize Virexo monorepo foundation"
```

Phase 2 — Backend Core

/virexo-phase

Phase 2: implement the backend application foundation.

Add:

- environment validation
- MongoDB connection lifecycle
- Express app/server separation
- centralized error handling
- request ID middleware
- structured console logger
- Helmet
- CORS
- cookie parser
- JSON size limits
- 404 handling
- graceful shutdown
- API response conventions
- health and readiness endpoints

Add focused integration tests.

Do not add feature models or authentication.

Commit:

```
git add .  
git commit -m "feat(api): establish secure backend foundation"
```

Phase 3 — Frontend Foundation and Design System

/virexo-phase

Phase 3: implement the frontend architecture and design system.

Add:

- routing foundation
- QueryClient
- Axios client
- Zustand application preferences store
- light/dark/system themes
- Tailwind design tokens
- reusable Button, Input, Avatar, Modal, Dropdown, Spinner, Skeleton, Toast and EmptyState
- responsive public and application layouts
- error boundary
- 404 page
- reduced-motion support

Create only placeholder routes for future features.

Verify desktop and mobile layouts in the browser.

Commit:

```
git add .  
git commit -m "feat(web): create responsive design system and app shell"
```

Phase 4 — Authentication Backend

```
/virexo-phase
```

Phase 4: implement core backend authentication.

Add:

- User model
- Session or RefreshToken model
- signup
- login
- refresh
- logout current session
- logout all sessions
- get current user
- bcrypt password hashing
- short-lived access JWT
- rotating refresh tokens
- hashed refresh-token storage
- token reuse detection
- remember-me refresh expiry
- auth validation
- auth rate limiting
- authentication middleware
- integration tests

Do not implement email verification or password reset yet.

Commit:

```
git add .  
git commit -m "feat(auth): implement secure token-based authentication"
```

Phase 5 — Email Verification and Password Recovery

```
/virexo-phase
```

Phase 5: implement account email workflows.

Add:

- provider-independent email service
- Brevo adapter

- console email adapter for local development
- verification token model or secure hashed-token fields
- send verification email
- verify email
- resend verification with cooldown
- forgot password
- reset password
- single-use expiring tokens
- revoke all sessions after password reset
- HTML and plain-text templates
- tests

Never expose token values in logs.

Document manual Brevo setup and environment variables.

Commit:

```
git add .  
git commit -m "feat(auth): add email verification and password recovery"
```

Phase 6 — Authentication Frontend

/virexo-phase

Phase 6: build and connect frontend authentication.

Add:

- signup
- login
- remember me
- verify-email result page
- resend verification
- forgot password
- reset password
- authenticated route guard
- guest route guard
- in-memory access-token handling
- silent refresh
- Axios refresh interceptor with request deduplication
- logout and logout-all-sessions UI
- form validation and accessible errors
- loading and success states

Do not store access or refresh tokens in localStorage.

Add focused component and integration tests.

Commit:

```
git add .  
git commit -m "feat(web-auth): implement complete authentication experience"
```

Phase 7 — User Profile, Privacy and Preferences

/virexo-phase

Phase 7: implement user profile and settings.

Backend:

- unique username
- display name
- bio
- avatar metadata
- last seen
- privacy settings
- notification settings
- profile update APIs
- username availability/search-safe fields
- authorization and validation

Frontend:

- profile page
- settings layout
- account, appearance, privacy and notification sections
- avatar preview with temporary local preview only
- optimistic updates where safe

Actual Cloudinary upload belongs to the media phase.

Commit:

```
git add .  
git commit -m "feat(users): add profiles privacy and preferences"
```

Phase 8 — Conversation Domain

```
/virexo-phase
```

Phase 8: implement one-to-one and group conversation data architecture.

Add:

- Conversation model
- ConversationMember model only if justified by the documented schema
- direct-chat uniqueness
- group owner/admin/member roles
- create or retrieve direct conversation
- create group
- list conversations with cursor pagination
- conversation details
- rename group
- update group avatar metadata
- add/remove members
- promote/demote admins
- leave group
- transfer ownership when required
- authorization tests

Do not implement messages or sockets.

Commit:

```
git add .  
git commit -m "feat(conversations): implement direct and group conversation  
domain"
```

Phase 9 — Message REST Domain

```
/virexo-phase
```

Phase 9: implement the core message REST domain.

Add:

- Message model and indexes
- text messages
- server-generated message identifiers
- conversation authorization

- create message
- cursor-based message history
- stable chronological ordering
- duplicate-request idempotency key
- soft deletion foundations
- message DTOs
- conversation last-message projection
- unread-count foundations
- validation, rate limits and tests

Do not add Socket.IO or advanced message operations.

Commit:

```
git add .  
git commit -m "feat(messages): implement paginated message domain"
```

Phase 10 — Socket.IO Foundation and Presence

/virexo-phase

Phase 10: implement the authenticated real-time layer.

Add:

- Socket.IO server bootstrap
- JWT socket authentication
- user rooms
- conversation rooms
- explicit event contracts in packages/shared
- connect/disconnect handling
- online presence
- last-seen persistence with write throttling
- multi-tab connection counting
- typing start/stop with expiry
- socket error acknowledgements
- connection cleanup
- tests where practical

Design for one Render instance and document the future Redis adapter boundary.

Commit:

```
git add .
git commit -m "feat(realtime): add authenticated sockets presence and typing"
```

Phase 11 — Delivery and Read State

/virexo-phase

Phase 11: implement message lifecycle state.

Add:

- sent, delivered and read semantics
- per-user read cursor or receipt model selected in the schema
- delivered acknowledgements
- mark conversation read
- unread counts
- real-time receipt events
- reconnect synchronization
- duplicate-event safety
- last-read indicators
- privacy-aware read receipts
- tests for multi-user and multi-device cases

Avoid creating one database document per receipt unless the architecture requires it.

Commit:

```
git add .
git commit -m "feat(messages): add delivery read receipts and unread state"
```

Phase 12 — Main Chat Interface

/virexo-phase

Phase 12: build the connected chat interface.

Add:

- responsive three-region desktop layout
- mobile route-based navigation

- conversation list
- chat header
- virtualized or efficient message list
- cursor-based upward pagination
- message composer
- real-time incoming messages
- optimistic sending with failed/retry state
- typing indicator
- online and last-seen display
- sent/delivered/read icons
- unread badges
- auto-scroll rules
- date separators
- empty, loading and error states

Preserve scroll position when loading older messages.
Verify mobile and desktop in browser.

Commit:

```
git add .  
git commit -m "feat(chat-ui): build responsive real-time messaging interface"
```

Phase 13 — Advanced Message Actions

/virexo-phase

Phase 13: implement advanced message operations.

Add:

- edit message with edited indicator
- delete for self
- delete for everyone with time/permission policy
- reply
- forward
- pin/unpin
- reactions
- message action menu
- backend authorization
- audit-safe message state
- real-time synchronization
- optimistic updates with rollback
- tests

Do not implement media or search.

Commit:

```
git add .  
git commit -m "feat(messages): add editing deletion replies pins and reactions"
```

Phase 14 — Media, Files and Voice Notes

/virexo-phase

Phase 14: implement secure media messaging.

Add:

- Cloudinary service adapter
- Multer memory storage
- signed/controlled upload API
- image, video, PDF and general file validation
- file-size and MIME allowlists
- upload progress
- attachment message schema
- image preview
- video player
- PDF/file cards
- secure download behavior
- browser MediaRecorder voice notes
- recording timer, cancel, preview and upload
- duration metadata
- audio player
- Cloudinary deletion when authorized
- tests

Never write uploads permanently to local disk.
Document Cloudinary setup.

Commit:

```
git add .  
git commit -m "feat(media): add attachments and voice messages"
```

Phase 15 — Search

```
/virexo-phase
```

Phase 15: implement search without paid Atlas Search.

Add:

- indexed user search
- conversation search
- MongoDB text-index message search
- filters by conversation, sender, date and attachment type
- cursor pagination
- safe query limits
- debounced frontend search
- keyboard navigation
- result highlighting without unsafe HTML
- jump-to-message behavior
- search rate limits
- tests

Document Free Atlas search limitations.

Commit:

```
git add .  
git commit -m "feat(search): add user conversation and message search"
```

Phase 16 — Notifications

```
/virexo-phase
```

Phase 16: implement notifications.

Add:

- Notification model
- real-time in-app notifications
- unread notification count
- notification list and pagination
- mark one/all as read
- group invitations and role-change notifications
- reactions, replies and mentions where applicable
- notification preferences

- Web Notifications API for active browser sessions
- duplicate-notification prevention
- sound preference with accessible defaults
- tests

Do not depend on a paid push service.

Commit:

```
git add .
git commit -m "feat(notifications): add real-time notification system"
```

Phase 17 — Reports and Administration

/virexo-phase

Phase 17: implement moderation and administration.

Add:

- report user
- report message
- report conversation
- Report model
- duplicate-report controls
- admin role and authorization middleware
- admin user listing and search
- suspend/restore user
- review reports
- report status and moderator notes
- immutable admin audit log
- protected admin interface
- confirmation dialogs
- pagination and filters
- tests proving non-admin denial

Do not expose sensitive authentication data to admins.

Commit:

```
git add .
git commit -m "feat(admin): add reporting moderation and user management"
```

Phase 18 — Security and Reliability Audit

```
/virexo-phase
```

Phase 18: perform a focused security and reliability hardening pass.

Audit and fix:

- authentication and authorization boundaries
- token rotation and reuse handling
- CORS and cookies across Vercel and Render
- NoSQL injection risks
- XSS and unsafe rendering
- upload abuse
- rate limiting
- pagination limits
- mass assignment
- sensitive logging
- account enumeration
- socket authorization
- stale room membership
- reconnect races
- duplicate message creation
- graceful shutdown
- database indexes
- dependency vulnerabilities

Add regression tests for every fixed issue.

Do not make cosmetic refactors.

Commit:

```
git add .  
git commit -m "security: harden authentication APIs uploads and sockets"
```

Phase 19 — Performance and Accessibility

```
/virexo-phase
```

Phase 19: optimize measurable performance and accessibility.

Add or improve:

- route-level lazy loading
- query cache policies
- render profiling fixes
- message list performance
- image dimensions and lazy loading
- bundle analysis
- Mongo query projections
- lean reads where appropriate
- database index verification
- keyboard navigation
- focus management
- semantic labels
- contrast
- reduced motion
- screen-reader announcements for important chat events

Run Lighthouse and document results.
Avoid premature micro-optimizations.

Commit:

```
git add .  
git commit -m "perf: improve Virexo performance and accessibility"
```

Phase 20 — Complete Testing

/virexo-phase

Phase 20: complete the automated test suite.

Add:

- backend unit tests for critical services
- API integration tests
- authorization matrix tests
- refresh-token rotation tests
- socket integration tests
- frontend component tests
- authentication flow tests
- chat state tests
- Playwright smoke tests for signup/login/chat/group/settings/admin
- test fixtures and cleanup
- root test commands
- CI-safe environment handling

Prioritize critical behavior over line-count coverage.
All tests, lint and production builds must pass.

Commit:

```
git add .  
git commit -m "test: add critical end-to-end and integration coverage"
```

Phase 21 — Documentation

/virexo-phase

Phase 21: complete project documentation.

Create or update:

- README.md
- docs/FOLDER_STRUCTURE.md
- docs/API.md or OpenAPI specification
- docs/SOCKET_EVENTS.md
- docs/ENVIRONMENT_VARIABLES.md
- docs/DATABASE_SCHEMA.md
- docs/SECURITY_MODEL.md
- docs/DEPLOYMENT.md
- docs/TROUBLESHOOTING.md
- docs/FREE_TIER_LIMITATIONS.md
- Mermaid architecture diagram
- authentication sequence diagram
- real-time message sequence diagram

Include exact local setup commands without real credentials.
Ensure documentation matches current code.

Commit:

```
git add .  
git commit -m "docs: complete Virexo engineering documentation"
```

Phase 22 — Deployment Configuration

```
/virexo-phase
```

Phase 22: prepare Virexo for free-tier deployment.

Add:

- production environment validation
- Vercel SPA rewrite configuration
- Render build/start configuration
- production CORS allowlist
- cross-site secure cookie configuration
- frontend API and Socket.IO environment URLs
- health checks
- deployment-safe logging
- Cloudinary configuration validation
- MongoDB Atlas configuration documentation
- Brevo configuration documentation
- post-deployment verification script or checklist

Do not insert real secrets.

Run local production builds.

Commit:

```
git add .  
git commit -m "chore: configure Virexo production deployment"
```

Phase 23 — Manual Deployment

Use this prompt after connecting GitHub to the services:

```
/virexo-phase
```

Phase 23: guide me through manual deployment and verify the deployed app.

Do not change code unless deployment exposes a real defect.

Walk through:

1. MongoDB Atlas Free cluster
2. Cloudinary Free account
3. Brevo Free transactional email

4. Render Free Web Service for apps/api
5. Vercel Hobby project for apps/web
6. production environment variables
7. allowed frontend/backend origins
8. secure cookies
9. Socket.IO connection
10. Cloudinary upload
11. verification/reset email
12. complete smoke test

For every service provide:

- exact dashboard section
- field name
- value source
- safe placeholder
- expected result
- debugging check

Never request that I paste secrets into chat.

Commit deployment fixes only when required:

```
git add .  
git commit -m "fix: resolve production deployment issues"
```

Phase 24 — Final Release Audit

/virexo-phase

Phase 24: perform the final release audit.

Check:

- all required features
- frontend and backend production builds
- automated tests
- API authorization
- Socket.IO behavior
- responsive UI
- accessibility
- broken routes
- environment documentation
- leaked secrets
- repository cleanliness

- free-tier compatibility
- deployed smoke tests
- README accuracy

Create:

- docs/RELEASE_CHECKLIST.md
- docs/KNOWN_LIMITATIONS.md

Fix only release-blocking defects.

Return a pass/fail checklist and remaining limitations.

Final commit:

```
git add .  
git commit -m "release: prepare Virexo portfolio production build"  
git tag v1.0.0
```

Token-Saving Operating Rules

1. Use one conversation per phase.
2. Keep the Workspace Rule Always On.
3. Never paste the complete project specification again.
4. Reference `docs/BUILD_STATUS.md`.
5. Ask Antigravity to edit files directly.
6. Do not ask it to print full code.
7. Do not request broad refactoring.
8. Keep one objective per prompt.
9. Require tests only for changed behavior.
10. Commit after every successful phase.
11. Use `git diff` instead of asking for code recaps.
12. Start a new conversation after a failed or confused agent run.

Recovery Prompt

Use this when a phase fails:

Read the current repository, workspace rule, BUILD_STATUS and latest git diff.

The previous phase is incomplete.

Do only this:

1. identify the first root-cause failure,

2. fix it with the smallest change,
3. rerun the failing command,
4. rerun relevant regression checks,
5. update BUILD_STATUS,
6. stop.

Do not regenerate working files or add new features.

Code-Review Prompt

Run after important phases:

Review only the latest git diff.

Check correctness, security, authorization, race conditions, data consistency, accessibility and unnecessary complexity.

Do not edit code initially.

Return only:

1. blocking issues
2. important issues
3. optional improvements
4. exact affected files

Maximum 400 words.

Then use:

Fix only the blocking and important issues from the review. Preserve existing architecture, run relevant tests, update BUILD_STATUS and stop.